



UNIVERSITÀ DI TRENTO

Relazione progetto Reti Neurali

Programmazione avanzata ed intelligenza artificiale

Studente: Daniele Pizzoli

Matricola: 243473

Corso: Programmazione avanzata ed
intelligenza artificiale
Docente: Prof. Paolo Rech

May 29, 2026

Contents

1	Introduzione	3
2	Scelta del task e criterio di valutazione	3
2.1	Perché una CNN per l'audio	3
3	Dataset e preprocessing	4
4	Architetture considerate	4
5	Addestramento e valutazione	5
6	Andamento dell'addestramento	5
7	Analisi delle prestazioni	8
7.1	Matrice di confusione di LightCNN	8
7.2	Matrice di confusione di AudioResNet	9
7.3	Matrice di confusione di AudioMobileNetV2	10
7.4	Matrice di confusione di AudioAlexNet	11
7.5	Analisi degli errori	11
7.6	Considerazioni sulla scelta del modello	12
8	Conclusioni	12

1 Introduzione

Il task scelto è la classificazione di comandi vocali a singola parola, quindi un problema di classificazione multiclass su segnali audio brevi e abbastanza controllati. Il dataset adottato è Google SpeechCommands v0.02 e le classi considerate sono dieci: `yes`, `no`, `up`, `down`, `left`, `right`, `on`, `off`, `stop` e `go`.

L'obiettivo del progetto non è soltanto ottenere una buona accuratezza, ma costruire una pipeline coerente dall'ingresso del segnale fino alla valutazione finale. Per questo il flusso di lavoro segue un'impostazione tipica dei sistemi audio moderni: l'audio grezzo viene normalizzato, trasformato in una rappresentazione tempo-frequenza e poi classificato da modelli neurali con complessità diversa. La discussione tiene insieme risultati numerici, qualità delle predizioni e scelte progettuali.

2 Scelta del task e criterio di valutazione

Il problema dei comandi vocali è adatto alla consegna perché consente di studiare un task reale, con un dataset ben definito e una separazione chiara tra training, validation e test set. Inoltre, la struttura del segnale audio rende naturale l'uso di reti convoluzionali, che possono lavorare sugli spettrogrammi come se fossero immagini.

La valutazione del sistema si basa principalmente su accuracy, matrice di confusione e analisi manuale dei campioni errati. Questa combinazione è più informativa della sola accuracy, perché permette di capire non solo quanto il modello sia corretto in media, ma anche quali classi vengano confuse e in quali condizioni. In un task audio di questo tipo, infatti, gli errori tra parole simili sono spesso più interessanti del punteggio complessivo, perché mostrano con chiarezza i limiti della rappresentazione e dell'architettura scelta.

2.1 Perché una CNN per l'audio

La scelta di una CNN è naturale perché, una volta trasformato il segnale in spettrogramma Mel, il problema assume molte caratteristiche di una classificazione su immagini. L'audio non viene più interpretato come una sequenza grezza di campioni, ma come una rappresentazione tempo-frequenza ottenuta con la STFT e compressa nello spazio di Mel. Il risultato è una matrice bidimensionale in cui l'asse orizzontale rappresenta il tempo e l'asse verticale la distribuzione dell'energia sulle frequenze.

Questa trasformazione è il punto chiave: la CNN può sfruttare filtri locali per individuare pattern ripetuti o significativi, senza dover modellare esplicitamente ogni istante della sequenza. In pratica, il problema si avvicina molto alla visione artificiale, perché il modello può cercare bordi, bande, transizioni e strutture armoniche che caratterizzano la pronuncia di una parola.

Un altro vantaggio importante è la condivisione dei pesi: lo stesso kernel viene applicato su tutta la rappresentazione, quindi la rete riesce a cercare gli stessi indizi acustici in posizioni diverse dello spettrogramma. Questo è utile sia rispetto al tempo, perché la stessa parola può comparire all'inizio o alla fine del clip, sia rispetto alla frequenza, perché una stessa pronuncia può apparire leggermente più in alto o più in basso in funzione della voce o delle condizioni di registrazione. Il pooling riduce la sensibilità a piccole traslazioni e rende il modello più robusto rispetto a variazioni di pronuncia, velocità e allineamento temporale.

Dal punto di vista delle feature, la CNN è particolarmente adatta a catturare pattern molto locali: una consonante esplosiva, una transizione tra vocali o una banda armonica stabile occupano regioni ristrette dello spettrogramma e possono essere riconosciute da kernel piccoli già nei primi strati. Gli strati successivi combinano queste informazioni elementari in rappresentazioni via via più astratte, fino a modellare la struttura complessiva della parola.

Infine, rispetto a RNN e LSTM, la CNN offre un vantaggio pratico importante: l'intera rappresentazione può essere elaborata in parallelo, mentre le reti ricorrenti devono scorrere il segnale passo dopo passo. Questo rende l'addestramento più veloce, più semplice da parallelizzare su GPU e più adatto a modelli leggeri o a scenari in cui il tempo di inferenza è importante. Per questo motivo, per un task come questo, una CNN costituisce un baseline solido e coerente con il tipo di dato in ingresso.

3 Dataset e preprocessing

Il dataset utilizzato è SpeechCommands v0.02, presente nella cartella `dataset_audio/SpeechCommands/`. La pipeline implementata in `dataset.py` filtra il dataset originale per mantenere soltanto le dieci classi di interesse e suddivide i campioni nei tre sottoinsiemi standard: training, validation e testing.

Ogni segnale audio viene portato a una frequenza di campionamento di 16 kHz e normalizzato a una durata fissa di un secondo. Questa scelta semplifica il training e rende l'input compatibile con il modello, che si aspetta tensori di dimensione costante. La conversione in spettrogramma Mel è gestita da `audio_utils.py`, dove l'audio viene trasformato con `MelSpectrogram` e convertito in dB tramite `AmplitudeToDB`. I parametri principali usati nella pipeline sono `N_FFT = 1024`, `HOP_LENGTH = 512` e `N_MELS = 64`.

In pratica, ogni clip viene prima adattata alla stessa lunghezza temporale e poi rappresentata come immagine monocanale, così da rendere applicabili architetture convoluzionali nate per la visione artificiale. Questo passaggio è centrale: per il problema considerato, lo spettrogramma conserva l'informazione utile della parola pronunciata molto meglio di una rappresentazione puramente numerica del segnale, perché preserva sia l'andamento temporale sia la distribuzione dell'energia sulle frequenze più rilevanti.

$$\text{spettrogramma_Mel} = \text{AmplitudeToDB}(\text{MelSpectrogram}(x))$$

4 Architetture considerate

Le architetture implementate nel progetto sono definite in `models.py`. Il baseline principale è **LightCNN**, una rete leggera con due blocchi convoluzionali e un classificatore fully connected finale. Questa soluzione è pensata per offrire un buon compromesso tra tempo di addestramento e capacità discriminativa, ed è una scelta coerente con un primo modello di riferimento.

Accanto al modello leggero è presente anche **AudioResNet**, ottenuto adattando una ResNet-18 all'ingresso monocanale degli spettrogrammi audio. La modifica principale consiste nella sostituzione del primo layer convoluzionale, che nella versione standard si aspetta tre canali RGB. Le architetture **AudioMobileNetV2** e **AudioAlexNet** completano il confronto sperimentale: la prima punta a un buon compromesso tra efficienza e

capacità rappresentativa, mentre la seconda rappresenta un backbone più pesante e più sensibile alla dimensione spaziale dell'input.

La differenza tra i modelli non riguarda solo il numero di parametri, ma anche il tipo di capacità rappresentativa. **LightCNN** consuma meno risorse ed è più rapida da valutare, mentre **ResNet** tende a sfruttare meglio la profondità dei layer per estrarre feature più stabili e più gerarchiche, a costo di un'elaborazione più pesante. **MobileNet** prova a mantenere buona accuratezza con un costo inferiore rispetto alle reti classiche, mentre **AlexNet** richiede un adattamento più marcato dell'input e in questo progetto risulta la soluzione meno efficace. In tutti i casi, convoluzioni, normalizzazione e pooling rispondono alla stessa esigenza: estrarre feature locali dal segnale audio e trasformarle in una rappresentazione progressivamente più astratta e più facile da classificare.

Il confronto tra i due modelli è coerente con la consegna del corso, perché permette di osservare come una rete più semplice possa essere sufficiente per un primo baseline, ma anche come una soluzione più profonda possa migliorare la separazione tra classi quando il problema diventa più ambiguo. In altre parole, la relazione non cerca un solo modello vincente, ma mostra il trade-off tra semplicità, costo computazionale e qualità della classificazione.

5 Addestramento e valutazione

L'addestramento è gestito da `train.py`, mentre la valutazione è eseguita da `evaluate.py`. Il processo di valutazione carica il checkpoint migliore salvato durante il training, esegue l'inferenza sul test set e produce due tipi di output: la matrice di confusione e un file CSV con i campioni errati. Quest'ultimo aspetto è particolarmente utile perché consente un'analisi qualitativa dei casi in cui il modello fallisce.

La funzione di loss usata nel problema è quella tipica della classificazione multi-classe, cioè la cross-entropy. In questo contesto il modello impara a massimizzare la probabilità della classe corretta a partire dallo spettrogramma in ingresso. Dal punto di vista sperimentale, la qualità finale non dipende solo dal numero di epoche o dal tasso di apprendimento, ma anche dalla stabilità della rappresentazione audio e dalla capacità della rete di distinguere comandi foneticamente simili.

6 Andamento dell'addestramento

I file `training_history_<model>.csv` registrano, per ogni epoca, `train_loss`, `validation_loss` e `validation_accuracy`. Il confronto tra i quattro log mostra che tutti i modelli convergono in modo regolare, ma con prestazioni differenti: **AudioResNet** raggiunge il miglior risultato finale con una validation accuracy del 94.17%, seguito da **AudioMobileNetV2** con 91.71%, **LightCNN** con 90.98% e **AudioAlexNet** con 89.36% come miglior valore osservato.

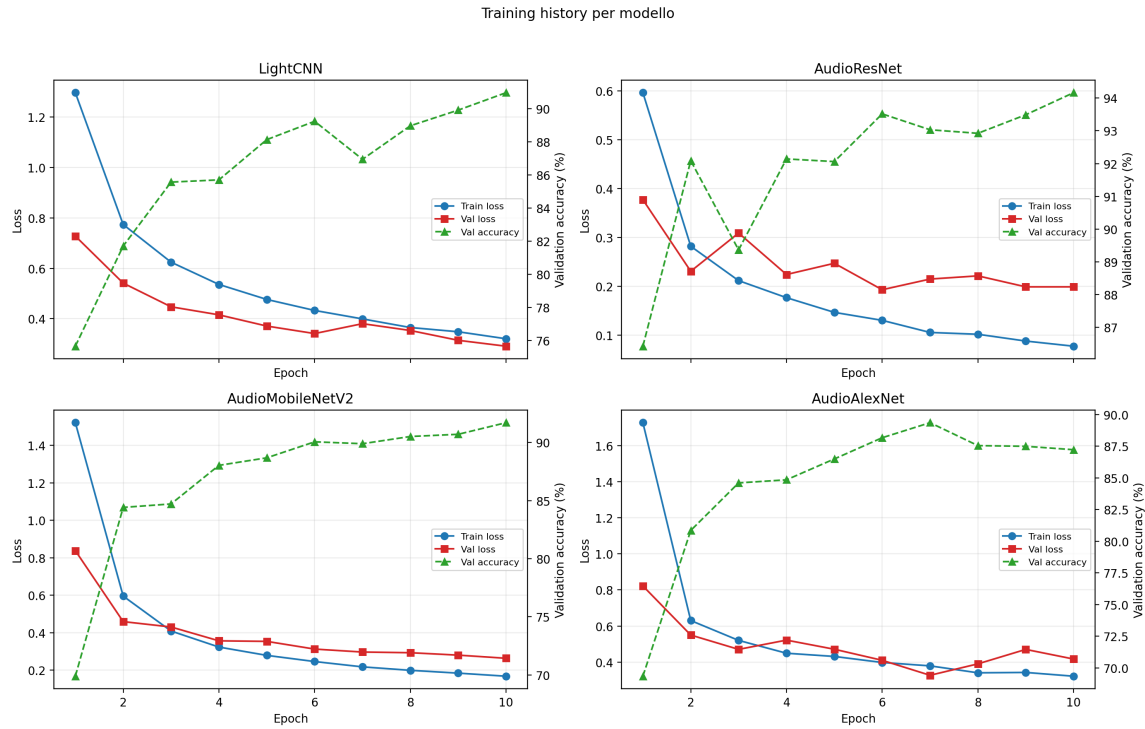


Figure 1: Andamento di training loss, validation loss e validation accuracy per ciascun modello.

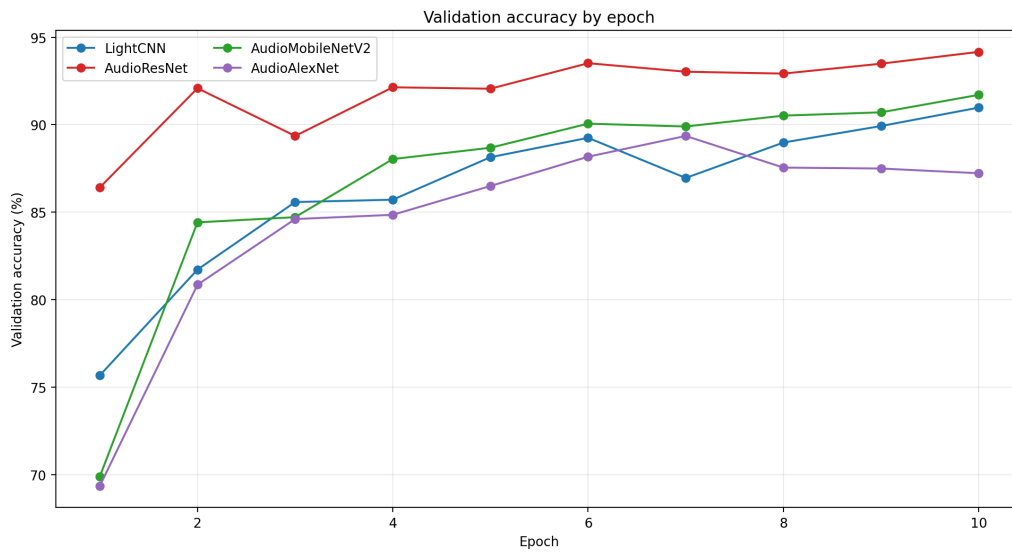


Figure 2: Confronto della validation accuracy tra i modelli addestrati.

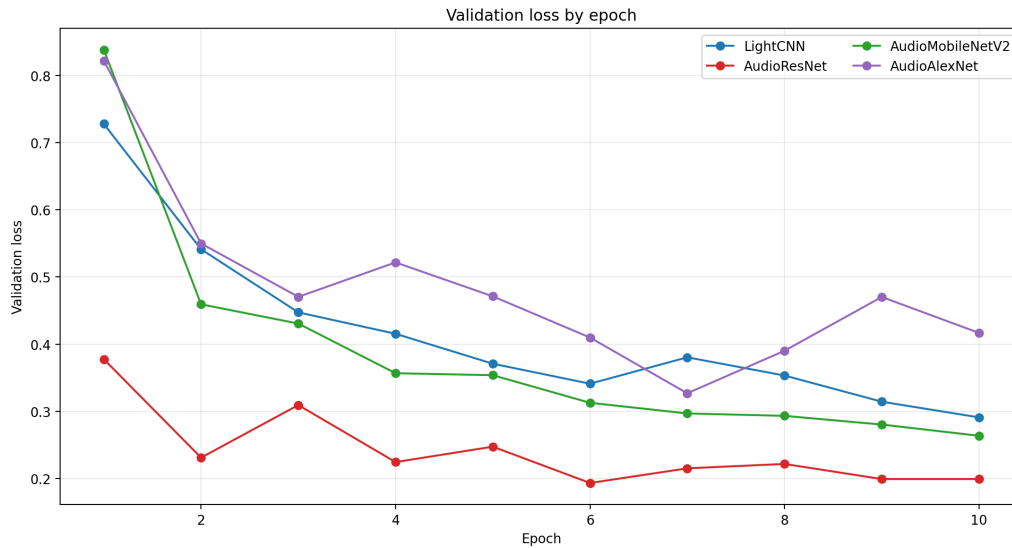


Figure 3: Confronto della validation loss tra i modelli addestrati.

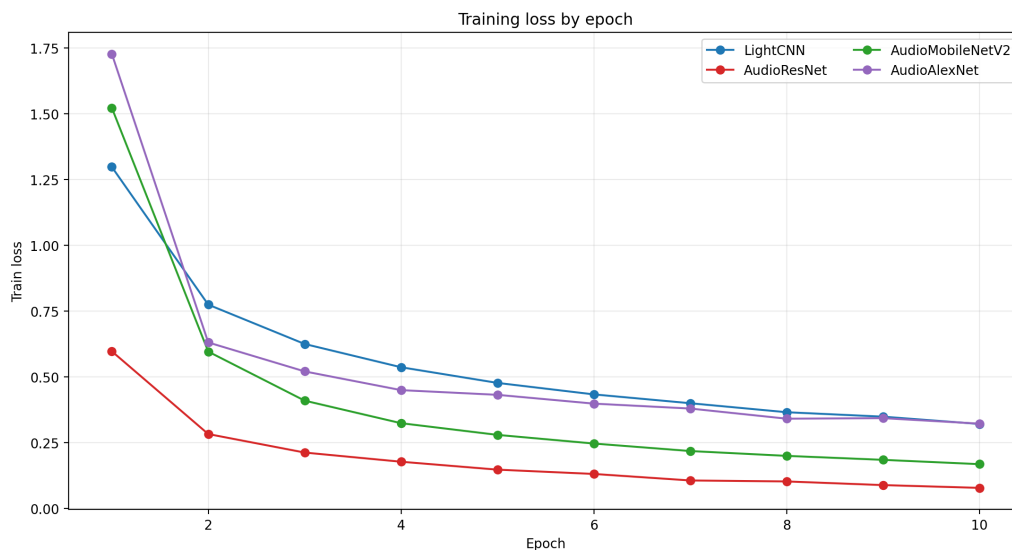


Figure 4: Confronto della training loss tra i modelli addestrati.

La Figura 1 permette di leggere in modo compatto sia la convergenza delle loss sia la stabilizzazione dell'accuracy nei diversi esperimenti. La Figura 2 rende invece più immediato il confronto tra i modelli e conferma che la ResNet è quella che generalizza meglio sul validation set. Le figure di validation loss e training loss chiariscono invece come i modelli più forti non solo raggiungano un'accuracy migliore, ma lo facciano anche con un andamento più stabile delle curve di ottimizzazione. L'andamento di AlexNet è più irregolare e si ferma prima degli altri, segno che questa architettura è più fragile rispetto alle dimensioni e alla natura dello spettrogramma audio usato in ingresso.

7 Analisi delle prestazioni

7.1 Matrice di confusione di LightCNN

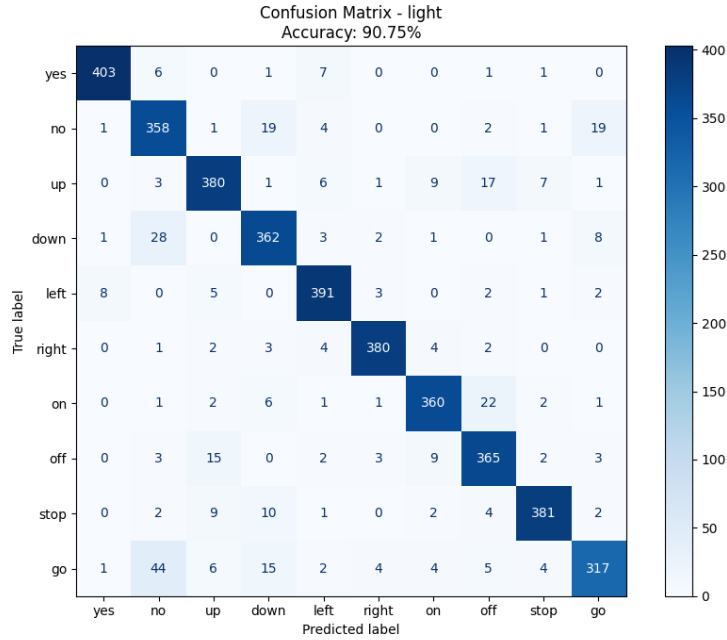


Figure 5: Matrice di confusione del modello LightCNN.

La matrice di confusione mostrata in Figura 5 evidenzia che il modello leggero riesce a riconoscere correttamente una parte consistente dei comandi, ma mostra ancora errori concentrati tra classi vicine dal punto di vista acustico. In particolare, i comandi brevi e molto simili tra loro tendono a essere scambiati con maggiore frequenza, segnale che la rete cattura bene le strutture più evidenti ma fatica nei casi ambigui.

7.2 Matrice di confusione di AudioResNet

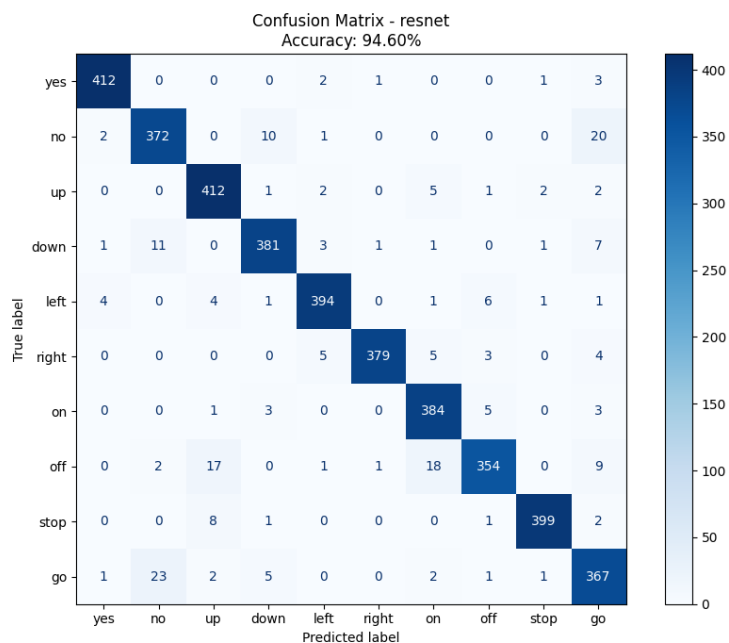


Figure 6: Matrice di confusione del modello AudioResNet.

Rispetto al modello leggero, la ResNet offre una rappresentazione più ricca e in genere produce una separazione più netta tra le classi. La Figura 6 mostra comunque che anche una rete più profonda non elimina del tutto le confusioni tra comandi simili, soprattutto quando la pronuncia è rapida o il segnale è meno netto. Il vantaggio principale della rete più profonda è una maggiore robustezza generale, mentre il limite rimane la naturale sovrapposizione acustica di alcune parole del dataset.

7.3 Matrice di confusione di AudioMobileNetV2

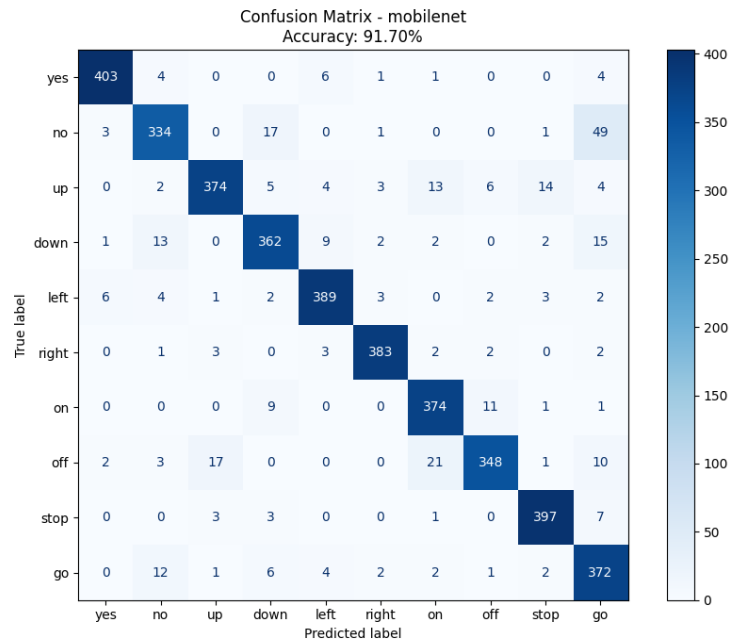


Figure 7: Matrice di confusione del modello AudioMobileNetV2.

AudioMobileNetV2 mostra un comportamento intermedio tra LightCNN e ResNet: il modello mantiene una buona capacità di discriminazione, ma alcune classi vicine restano ancora difficili da separare. Questa architettura conferma che una rete più efficiente non è necessariamente meno valida, ma richiede un input ben calibrato e una rappresentazione coerente per esprimere il suo vantaggio.

7.4 Matrice di confusione di AudioAlexNet

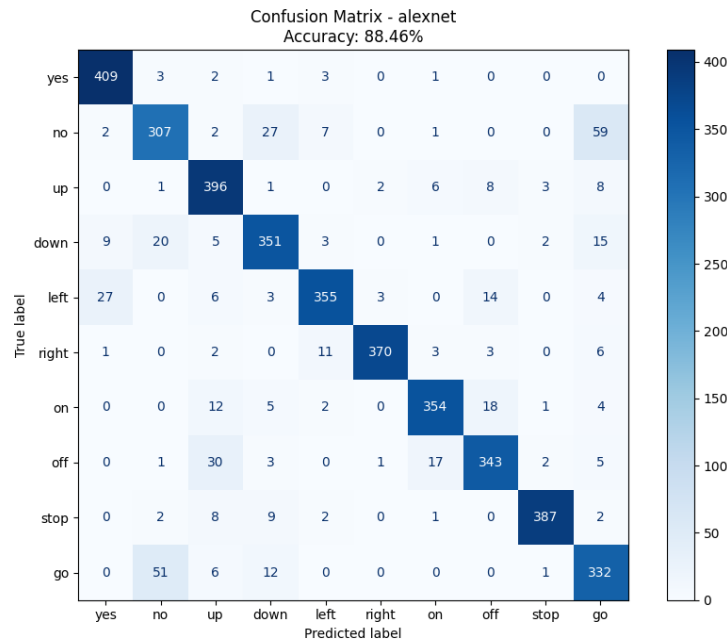


Figure 8: Matrice di confusione del modello AudioAlexNet.

La matrice di confusione di AudioAlexNet evidenzia un modello più sensibile alla complessità dell'input e meno stabile rispetto agli altri backbone. Il risultato è coerente con il comportamento osservato nei log di training, dove AlexNet raggiunge il suo massimo prima degli altri modelli e poi non migliora più in modo significativo.

7.5 Analisi degli errori

Per capire meglio il comportamento del classificatore, i campioni errati vengono salvati nella cartella `eval_out/wrong_samples/`. I file CSV `wrong_light.csv`, `wrong_resnet.csv`, `wrong_mobilenet.csv` e `wrong_alexnet.csv` permettono di ispezionare le predizioni sbagliate insieme alla confidenza associata alla classe predetta.

Questa analisi mette in evidenza un fenomeno ricorrente: molte delle errate classificazioni riguardano comandi foneticamente vicini oppure pronunciati in modo poco uniforme. Nei risultati osservati emergono errori frequenti tra `go`, `no`, `right`, `left`, `up`, `down`, `off` e `stop`. Il comportamento è coerente con la natura del dataset, in cui la variabilità tra parlanti, velocità di pronuncia e qualità della registrazione influisce molto sulla separabilità delle classi.

Questa parte della valutazione è importante anche dal punto di vista della consegna, perché mostra una lettura critica delle prestazioni: un modello può ottenere una buona accuracy complessiva e comunque fallire sistematicamente su alcune classi. Nel contesto del progetto questo aspetto conta più del valore assoluto dell'accuracy, perché dimostra la capacità di interpretare il modello e non soltanto di riportarne il punteggio.

7.6 Considerazioni sulla scelta del modello

Dal punto di vista pratico, LightCNN rappresenta una soluzione adatta quando si desidera un sistema semplice, veloce e facilmente addestrabile anche su hardware limitato. AudioResNet è il backbone con la migliore qualità finale, mentre AudioMobileNetV2 offre un compromesso interessante quando si vuole contenere il costo computazionale senza scendere troppo in accuratezza. AudioAlexNet, invece, è il modello meno convincente per questo dataset e conferma che non tutte le architetture nate per la visione si adattano allo stesso modo a una rappresentazione audio tempo-frequenza. In questo progetto il confronto tra i quattro approcci mostra bene il classico compromesso tra efficienza, robustezza e accuratezza.

8 Conclusioni

In conclusione, il progetto conferma che il riconoscimento di comandi vocali è un problema naturale per le reti neurali convoluzionali quando l'audio viene trasformato in spettrogrammi Mel. La pipeline implementata è coerente e modulare: il dataset viene filtrato, il test set resta separato dal training, la rappresentazione tempo-frequenza viene costruita in modo uniforme e i modelli vengono valutati sia quantitativamente sia qualitativamente.

I risultati ottenuti mostrano che il sistema è già funzionale, ma lascia spazio a ulteriori miglioramenti. Il confronto aggiornato tra LightCNN, ResNet, MobileNetV2 e AlexNet rende più chiaro il rapporto tra costo computazionale e qualità finale: la ResNet è la soluzione più forte, MobileNet mantiene un buon equilibrio, LightCNN resta il baseline più semplice e AlexNet risulta meno adatta a questo specifico input audio. Le direzioni più interessanti riguardano l'augmentation audio, una migliore calibrazione degli iperparametri, l'uso di architetture più sofisticate o di tecniche di ensemble e un'analisi ancora più accurata dei casi errati. In sintesi, la parte decisiva del progetto non è soltanto la scelta del modello, ma il bilanciamento tra preprocessing, complessità della rete, qualità dei dati di input e capacità di interpretare correttamente le prestazioni ottenute.